Workshop National EPSI — BAC+5 · Sprint du 19 au 23 octobre 2026

PROJECT RESILIENCE-ALPHA

Cyber-résilience et reprise autonome d'une plateforme de paiement fictive

Dossier d'ingénierie et de gouvernance — Consortium **G1**

Champ	Valeur
Livrable	`Workshop2026-M2-G1-Dossier.pdf` (§7.1 du sujet)
Dépôt	`Workshop2026-M2-G1-Code.zip` — README, `CAHIER-DES-CHARGES.md`, `ROADMAP.md`, `docs/`
Version	2.0 — 22 octobre 2026 (état du dépôt au gel du code, J4), complétée des mesures de recette
Périmètre	Données, comptes, transactions et entité **fictifs**. Actions offensives limitées au réseau `RESILIENCE-G1`.

*Ce dossier présente les décisions qui structurent le projet et renvoie au dépôt pour le détail. Chaque affirmation technique indique où la vérifier. Les chiffres marqués **[M]** attendent la série de rejeux de la recette J4 : ils ne sont pas annoncés tant qu'ils ne sont pas relevés.*

CAPELLE Yanis, FURCY Valentin, NECTOUX Louis, PALERMO Enzo, SERRE Aurelien

Table des matières

Cyber-résilience et reprise autonome d'une plateforme de paiement fictive 1
1. Mission et critère de réussite 1
2. Architecture et flux 1
2.1 Vue d'ensemble 1
2.2 Rôle des composants 1
2.3 Flux de la reprise autonome (chemin critique de la démo) 3
2.4 Décisions d'architecture (ADR) 3
3. Nature de l'incident : le banc d'essai Red Team 4
4. Détection : un modèle, pas des `if` 4
5. Remédiation autonome et privilèges maîtrisés 5
6. Supervision et observabilité 6
7. Éléments de sécurité 6
7.1 Consigne Additionnelle 1 : accès privilégié au back-office 7
8. Résultats des essais 8
9. Analyse des risques (EBIOS Risk Manager) 9
10. Objectifs de continuité, PCA/PRA 11
11. NIS2 / DORA et choix de gouvernance 12
12. Limites, et ce que nous ferions différemment en contexte industriel 13
13. Contribution de chacun 14
Annexe A — Où vérifier (dépôt `Workshop2026-M2-G1-Code.zip`) 16
Annexe B — Glossaire 16

myDiL epsi

1. Mission et critère de réussite

La Direction Nationale de la Sûreté Numérique (organisme fictif) veut vérifier qu'une plateforme de paiement peut **repérer seule un comportement anormal, décider d'une action corrective et la mener à bien**, puis prouver que le service est de nouveau utilisable. Le consortium G1 a conçu, développé et testé un démonstrateur conteneurisé qui joue ce scénario **sans intervention corrective humaine**.

Nous avons réduit le projet à un critère unique, qui gouverne tous les arbitrages :

> **La démo en cinq temps, jouable d'une seule commande, filmable en moins de 60 secondes sans coupure :** `avant` (transaction fictive OK) → `attaque` (depuis le PC Attaquant) → `détection` (modèle ML) → `remédiation` (automatique) → `après` (nouvelle transaction OK), avec la timeline et les indicateurs MTTD/MTTR affichés sur le dashboard.

Deux principes découlent du sujet et sont **codés dans le démonstrateur**, pas seulement écrits :

- **Pas de « conteneur qui redémarre » comme preuve.** La reprise n'est déclarée que lorsqu'une nouvelle transaction fictive traverse toute la chaîne (`scripts/verify_tx`, code de retour 0). Le moteur de remédiation boucle sur cette vérification.
- **Restaurer la disponibilité ≠ corriger une corruption.** Un pic de charge se traite en coupant la source et en redémarrant le service ; une altération de données se traite en restaurant un état sain. La table de décision refuse de se charger si ces deux familles sont mélangées (`remediation/decision.py`).

Les cinq domaines exigés sont présents et interconnectés : **Développement** (microservices, moteur de remédiation, dashboard), **Infrastructure** (Compose, Ansible, Prometheus/Grafana/Loki), **IA/Data** (détection ML, assistant RAG), **Cybersécurité** (Red Team, WAF, fonction SOC, SOPS, durcissement) et **Transformation** (EBIOS RM, PCA/PRA, NIS2/DORA). La **Consigne Additionnelle 1** (authentification renforcée du back-office par reconnaissance faciale et témoin LED ESP8266, §7.1) a été intégrée sans toucher à la boucle de reprise. Le fil conducteur est un **document de contrats d'interfaces** (`docs/contracts.md`) figé dès le premier jour : format de logs, métriques, événement d'anomalie, table de décision. Tout composant qui s'intègre doit le respecter, et des tests le vérifient.

2. Architecture et flux

2.1 Vue d'ensemble

Un seul PC Hôte (`192.168.137.1`, hotspot Windows `RESILIENCE-G1`) exécute l'ensemble de la plateforme sous Docker Compose ; le PC Attaquant, distinct, ne porte que les outils de test ; un PC IA optionnel peut héberger le modèle de langage.

```
PC Attaquant ──HTTPS 443──► [Caddy, mode « IP réelle »] ──► WAF (Nginx + ModSecurity + OWASP CRS)
[edge : SEUL exposé]
                                ├──► front (React/Vite/Tailwind)          [app]
                                ├──► payment-api ×3 répliques (FastAPI) ─┐
                                └──► auth (FastAPI, JWT) ──────────────────► PostgreSQL 16   [data :
interne]
Navigateur du PC Hôte ──127.0.0.1:9443──► WAF ──► back-office + /auth/* /admin/* ──► auth ──► face-id
                                                                          auth ──MQTT──► Mosquitto ──► ESP8266 (LED)

Observabilité [obs] : logs JSON ─► Promtail ─► Loki ; métriques ─► Prometheus ─► Grafana (règles SOC)
Détection : Prometheus (scrape 1 s) ─► collector (9 features / 1 s) ─► detector (IsolationForest) ─►
AnomalyEvent
Reprise   : moteur de remédiation [app, jamais exposé] ── table de décision
              ├─► docker-socket-proxy (restart, HUP)   ├─► Ansible (deny-list WAF, pg_restore)
              └─► verify_tx (preuve) ─► SSE ─► dashboard (timeline, MTTD/MTTR)
Chemin SOC : règles Prometheus ─► Alertmanager ─► POST /events/alertmanager (authentifié)
Assistant RAG (Ollama ≤ 8B + LangChain + Chroma) [ai] : lecture seule, réseau isolé de l'automate
```

2.2 Rôle des composants

Composant	Technologie	Réseau	Rôle
waf	Nginx + ModSecurity v3 + CRS 4 (image officielle, non-root, TLS au build)	edge, app	Point d'entrée unique (80/443), filtrage applicatif, **liste de blocage IP rechargeable à chaud**
waf-exporter	Python	obs	Transforme l'audit ModSecurity en métrique `resilience_waf_blocked_total{rule}` et en log `waf_blocked`
front	React, Vite, TypeScript, Tailwind	app	Interface « Banque Résilience » (écrans D1–D16) : comptes, virements, bénéficiaires, statistiques, cartes, virements programmés, notifications, relevés, confirmation renforcée des gros virements
payment-api ×3	FastAPI, SQLAlchemy	app, data	Comptes et transactions fictives (virement atomique, solde jamais négatif, plafonds, anti-double-dépense) ; 3 répliques réparties par le WAF (bascule sans coupure)
auth	FastAPI, bcrypt, JWT HS256 15 min	app, data	Connexion, rate-limit par IP resserrable à chaud ; parcours à étapes du back-office (identifiant → visage → mot de passe), rôles
back-office / face-id	React ; FastAPI + dlib	app, data	Habilitations des agents, reconnaissance faciale 1:1 (3 poses), gabarits chiffrés — joignable **uniquement depuis l'hôte** (§7.1)
mosquitto + ESP8266	broker durci, firmware C++	iot	Témoin LED bicolore de l'authentification (profil `iot`, facultatif)
postgres	PostgreSQL 16 (non-root, `cap_drop: ALL`)	data (interne)	Persistance ; jamais joignable depuis l'hôte
collector / detector	Python, scikit-learn	app, obs	Vecteur de features puis score d'anomalie ; émet l'événement d'anomalie (contrat §3)
remediation	FastAPI, ansible-runner	app, data	Décision (table YAML), actions, vérification, journal, timeline SSE ; **jamais sur `edge`, aucun port publié**
docker-socket-proxy	HAProxy	app	Seul accès à Docker : liste blanche de 5 couples méthode/chemin (ping, list, inspect, restart, `kill?signal=HUP`)
dashboard	React, Tailwind	app	Supervision : états des composants, score ML, timeline de reprise, MTTD/MTTR, IP bloquées (déblocage par jeton opérateur), onglet assistant
prometheus / loki / promtail / grafana / alertmanager	stack standard	obs (+ app pour Alertmanager)	Métriques, logs, visualisation, règles SOC, routage d'alertes vers la remédiation

Composant	Technologie	Réseau	Rôle
assistant	FastAPI, LangChain, Chroma, Ollama	ai	RAG sur la documentation et le journal d'incident, résumé DSI, post-mortem ; aucun privilège, aucune action

Les réseaux Compose séparent les usages : `edge` (seul le WAF), `app`, `data` (`internal: true`), `obs`, `ai` (assistant, sans intersection avec le moteur), `iot` (broker). Un audit automatisé (`security/hardening/audit.sh`) échoue si un autre service que le WAF publie un port, si le socket Docker est monté ailleurs qu'en lecture seule dans le proxy, ou si un conteneur est privilégié.

2.3 Flux de la reprise autonome (chemin critique de la démo)

1. **Avant** — `verify_tx` ouvre une session, résout deux comptes fictifs et poste un virement : code 0.
2. **Attaque** — `attacks/run.sh` depuis le PC Attaquant, intensité et durée plafonnées, kill-switch.
3. **Détection** — chaque seconde, le collector interroge Prometheus ; le detector score le vecteur ; au-delà du seuil il émet un `AnomalyEvent` (`incident_type`, score, features dominantes, requêtes de preuve) vers POST `/events`, authentifié. L'IP source est celle qui domine les journaux du WAF sur la fenêtre (≥ 60 %).
4. **Remédiation** — le moteur sélectionne la règle de la table de décision, exécute les actions dans l'ordre (blocage IP par playbook + rechargement du WAF, redémarrage via le proxy, resserrement du rate-limit, restauration du dernier dump), et **rejoue `verify_tx` après chaque action modifiante** (5 essais espacés de 3 s). Pour un flood déjà écrêté par la limitation de débit du WAF, le moteur **vérifie d'abord** : si `verify_tx` passe, l'incident est clos sans redémarrage inutile. Cooldown de 60 s, 3 tentatives au plus, puis escalade `notify_operator`.
5. **Après** — `verify_tx` OK → incident clos, MTTD/MTTR calculés, timeline publiée en SSE au dashboard.

`scripts/demo.sh` enchaîne ces cinq temps d'une commande ; il **n'exécute aucune action corrective** : il observe et prouve.

2.4 Décisions d'architecture (ADR)

Décision	Choix	Pourquoi
Langage backend	Python 3.12 + FastAPI partout (ADR 0001)	une chaîne d'outillage, synergie avec scikit-learn, lib partagée `resilience_common`
Accès Docker de l'automate	proxy HAProxy à liste blanche, jamais le socket brut (ADR 0002)	un socket brut équivaut à root sur l'hôte ; les variables génériques des proxys du marché ouvrent `containers/create`
Fonction SOC	règles Grafana/Loki + miroir Prometheus → Alertmanager (ADR 0003)	Elastic/Wazuh écartés pour la charge mémoire (16 Go) ; même chaîne d'incident
Attaque	PC Attaquant physique pour la démo notée ; profil Compose `attack` en répétition	exigence du sujet ; reproductibilité
Déclenchement	détecteur ML = chemin principal ; SOC = chemin secondaire, dédoublonné par le moteur	la décision vient du modèle (sujet §3.3) ; les règles seuil sont un filet

Décision	Choix	Pourquoi
Back-office (Consigne 1)	app dédiée, caméra lue par le navigateur de l'hôte, LED via MQTT, listener réservé à l'hôte, photos chiffrées (ADR 0004–0007)	le facteur visage ne protège que l'accès privilégié ; la reprise autonome n'en dépend jamais

3. Nature de l'incident : le banc d'essai Red Team

Le périmètre offensif est strict : cible exclusive = la stack du projet sur `RESILIENCE-G1`, données fictives, durée bornée (≤ 60 s), intensité plafonnée (≤ 100 req/s), arrêt automatique, kill-switch (`run.sh --stop`) et remise à l'état « avant » (`reset.sh`). Chaque scénario est un **contrat Red Team ↔ Blue Team** : il déclare le signal attendu, l'`incident_type` et la remédiation attendue, et un validateur vérifie la cohérence avec la table de décision.

Scénario	Mode opératoire (ATT&CK)	Signal observable	Réponse attendue	Famille
service_flood (scénario principal)	saturation applicative (T1499.002) : rafale POST `/api/v1/transactions`, ≤ 80 req/s, 30 s	requêtes/s, ratio 5xx, latence p95 ↑	blocage de l'IP source + redémarrage de `payment-api`	**Disponibilité**
auth_abuse	credential stuffing (T1110.004) sur `/api/v1/login`	échecs de login/s, rate-limit	blocage IP + rate-limit × 0,5	Disponibilité / accès
web_attack	exploitation d'application exposée (T1190) : charges SQLi / traversée de chemin	blocages CRS/s	confirmation (Loki) + liste noire permanente	Confinement
data_corruption	manipulation de données stockées (T1565.001) : renommage réversible d'une table fictive, déclenché depuis le PC Hôte	erreurs DB/s, `verify_tx` KO	**restauration du dernier dump**, rien d'autre	**Intégrité**

L'attaque principale de la démo est `service_flood` : crédible pour une plateforme de paiement, reproductible, compatible avec une réponse automatique visible en moins d'une minute. `data_corruption` est le contre-exemple qui montre que l'équipe distingue bien **compromission** et **indisponibilité** : sa remédiation est une restauration, et le moteur refuse toute règle qui y mettrait un simple redémarrage. Des pannes **non malveillantes** (crash, gel d'un conteneur) complètent le banc (`attacks/chaos`) pour éprouver la résilience intrinsèque.

4. Détection : un modèle, pas des `if`

Données. Prometheus fournit les métriques du contrat (requêtes, latence, codes HTTP, échecs d'authentification, blocages WAF, CPU/RAM, erreurs DB), relevées **chaque seconde** sur les services. Le collector les agrège chaque seconde (`rate[5s]`) en un **vecteur de 9 features** : `req_per_s`, `p95_latency_ms`, `ratio_4xx`, `ratio_5xx`, `auth_failed_per_s`, `waf_blocked_per_s`, `cpu_pct_max`, `mem_pct_max`, `db_errors_per_s`. La définition du vecteur est unique et partagée (`ml/shared/features.py`) entre entraînement, collecte et inférence ; le detector attend chaque nouvelle fenêtre (pas de relève décalée).

Préparation. Le modèle est entraîné sur des **séries réelles capturées sur la stack** (21/10) : 3 601 fenêtres de trafic nominal (60 min, générateur de trafic légitime) et 3 passages de chacun des 4 scénarios. Les features sont standardisées (z-score avec plancher d'écart-type). Une première version, entraînée sur du synthétique, a servi au câblage. Mesurée sur la stack, la chaîne détectait en 25 à 37 s (p95) et routait mal le flood : nous avons accéléré la collecte (scrape et fenêtres d'une seconde) et ré-entraîné sur ces captures réelles.

Modèle. `IsolationForest` (scikit-learn, 200 arbres), non supervisé : il apprend la « forme » du normal et attribue un score d'anomalie. La calibration (`ml/training/calibrate.py`) exige que le score **tienne sur 4 fenêtres** (les pics normaux sont brefs) ; elle donne un seuil de 0,708 avec **0 faux positif sur les 3 601 fenêtres normales** et 12 passages d'attaque sur 12 détectés. Pour la démo, l'équipe a **abaissé le seuil à 0,65** (`ANOMALY_THRESHOLD`, surchargeable) afin de voir les attaques à faible débit du PC Attaquant, au prix d'environ 2 fausses alertes par heure mesurées sur la baseline. L'artefact est versionné (`v1.2.0`), entraîné au build de l'image, et son empreinte SHA-256 est vérifiée au chargement (`joblib` désérialise du code : on refuse un modèle altéré).

Résultats de calibration : délai modèle au pire 9 s (`service_flood`), 7 s (`auth_abuse`), 4 s (`web_attack`), 9 s (`data_corruption`). `web_attack` et `data_corruption` ne sont pas séparables par le seul score (signaux nuls au normal, où IsolationForest est aveugle) : un **garde-fou statistique** ($z \geq 15$ sur la feature dominante) les rattrape. Sur la stack, après réparation de la chaîne, un flood est détecté **en 2 s**. Le detector expose `resilience_anomaly_score` et émet l'`AnomalyEvent` du contrat avec les features dominantes, la requête PromQL de preuve et **l'IP source** (celle qui domine les journaux du WAF, ≥ 60 % des lignes, réseaux internes exclus).

Limites assumées. (1) Le modèle a été calibré **avant** le passage à 3 répliques de `payment-api` et la limitation de débit du WAF, qui changent la distribution des features : une recapture est ouverte (ROADMAP T1). (2) Le **type** d'incident est attribué par une heuristique de routage sur les features dominantes (`classify_incident`), pas par le modèle : c'est le modèle qui décide *qu'il y a* une anomalie, la table de décision qui décide *quoi faire*. (3) Un faux positif coûte au pire un blocage d'IP (expirant) et un redémarrage progressif d'une réplique, borné par le cooldown.

5. Remédiation autonome et privilèges maîtrisés

Décision. `remediation/decision_table.yml` est la seule source de vérité (schéma `docs/contracts.md` §4). Le moteur ne contient aucune règle métier codée en dur, uniquement des **invariants de sécurité** vérifiés au chargement : `data_corruption` ne contient jamais `restart/recreate` ; `restore_pg_dump` n'apparaît dans aucune autre règle ; la règle unknown ne contient aucune action destructive (rapport à blanc puis escalade).

incident_type	Actions (ordre)	Vérification	Escalade
service_flood	*vérification d'abord*, puis `waf_block_ip` → `restart_service(payment-api)`	`verify_tx`	`notify_operator`
auth_abuse	`waf_block_ip` → `tighten_rate_limit(auth, ×0,5)`	`verify_tx`	idem
web_attack	`waf_blacklist_ip` → `confirm_waf_block` (Loki)	`verify_tx`	idem
data_corruption	`restore_pg_dump(latest)` **seul**	`verify_tx`	idem
unknown	`dry_run_report`	`verify_tx`	idem

Exécution. Tout se passe **dans le conteneur** du moteur, jamais par `docker exec` : les playbooks Ansible (`waf_deny_ip.yml`, `pg_restore.yml`) y sont embarqués et exécutés sur `localhost` ; la deny-list du WAF est un volume partagé ; le WAF est rechargé par un signal HUP envoyé via le proxy ; la restauration passe par un client PostgreSQL sur le réseau `data` (`pg_restore --clean --single-transaction`, intégrité du dump contrôlée avant). `verify_tx` est embarqué et rejoué à travers le WAF, comme un vrai client.

Garde-fous. Idempotence (un événement rejoué n'ouvre pas deux incidents), incident actif dédoublonné, cooldown 60 s, 3 tentatives, `dry_run` forçable par l'environnement, liste blanche de services redémarrables (`payment-api`, `auth`, `front`), liste blanche de playbooks, IP et réseaux jamais bloqués (hôte, loopback, réseaux internes Docker : le moteur ne peut pas bloquer sa propre vérification), blocages WAF **expirants** (1 h par défaut, `REMEDIATION_WAF_BLOCK_TTL_S`), redémarrage **progressif** des répliques, validation stricte de toute valeur qui finit dans une configuration (une `source_ip` doit être une adresse littérale).

Privilèges (point sondé par le jury). Le socket Docker n'est monté qu'en lecture seule dans le proxy HAProxy, qui n'autorise que 5 couples méthode/chemin ; le moteur est authentifié par un Bearer interne (secret SOPS, comparaison à temps constant, *fail closed* si absent), n'est sur aucun réseau exposé et ne publie aucun port ; le conteneur est non-root, `cap_drop: ALL`, `no-new-privileges`. Un token volé permettrait au pire des redémarrages de trois services et l'ajout d'IP en deny-list, tracés dans le journal. Le **déblocage** d'une IP depuis le dashboard exige un **jeton opérateur distinct**, saisi par l'humain et jamais injecté par le proxy du dashboard.

Deuxième chemin de déclenchement. Quatre règles SOC Prometheus (miroir des règles Grafana) sont routées par Alertmanager vers `POST /events/alertmanager`, authentifié ; un adaptateur traduit le webhook en `AnomalyEvent` et le moteur dédoublonne. Ce chemin est **secondaire** : la décision reste au modèle.

6. Supervision et observabilité

- **Logs** : tous les services émettent du JSON sur une ligne (`ts`, `level`, `service`, `event`, `message`, `request_id`, champs contextuels) avec des **événements normalisés** (`transaction_created`, `auth_login_failed`, `waf_blocked`, `anomaly_detected`, `remediation_started/action/verified/failed`…) ; Promtail les pousse dans Loki.
- **Métriques** : préfixe `resilience_`, noms figés par le contrat ; MTTD et MTTR sont publiés par incident (`resilience_mttd_seconds`, `resilience_mttr_seconds`).
- **Fonction SOC** : 4 règles d'alerte couvrant les 4 `incident_type`, dans Grafana (affichage) et Prometheus (routage) ; cohérence vérifiée par test.
- **Dashboard** : états des composants (dont les 3 répliques), score ML lu dans Prometheus, timeline temps réel de la reprise (SSE : détection → décision → action → vérification → clôture), MTTD/MTTR, liste des IP bloquées par le WAF. Le navigateur ne parle qu'au nginx du dashboard (même origine) qui injecte le Bearer côté serveur : aucun secret dans le front.
- **Assistant RAG** : un onglet du dashboard pour interroger la documentation et le journal d'incident (Ollama, modèle ≤ 8B, Chroma), produire un **résumé pour la DSI** et un **post-mortem sans blâme** (chronologie générée de façon déterministe). Il peut *proposer* une commande opérateur (liste blanche, confirmation humaine) ; il n'agit jamais, et son réseau `ai` n'a aucune intersection avec le moteur.

7. Éléments de sécurité

Mesure	Mise en œuvre	Où vérifier
WAF, seul point exposé	image officielle OWASP CRS épinglée, non-root (8080/8443 publiés en	`security/waf/`, 25 tests de contrat

Mesure	Mise en œuvre	Où vérifier
	80/443), TLS généré au build, paranoïa 1, deny-list IP à chaud	
Segmentation	réseaux par usage, `data` interne, aucun `ports:` hors WAF (et broker MQTT lié à l'IP du hotspot, jamais `0.0.0.0`) ; en mode « IP réelle », Caddy écoute sur le hotspot et la boucle locale seulement	`infra/compose`, `infra/caddy`, `audit.sh` §5–7
Secrets	**SOPS + age** : secrets chiffrés dans le dépôt, clé privée hors git, `.env` déchiffré ignoré ; `gitleaks` en pre-commit ; token Alertmanager via secret Compose ; re-scan de l'archive (noms et contenu)	`security/sops/`, `scripts/package_deliverables.sh`
Durcissement	images slim, utilisateurs non-root, `cap_drop: ALL`, `no-new-privileges`, `read_only` + tmpfs, limites CPU/RAM, healthchecks ; Postgres démarre non-root sans aucune capability ajoutée	`security/hardening/hardening.md`, tests `infra/tests`
Authentification	bcrypt, JWT 15 min à algorithme épinglé, rate-limit `/login` (en-tête `X-Forwarded-For` cru seulement depuis le WAF) ; **confirmation par mot de passe des gros virements** (step-up) ; API internes par tokens dédiés	`services/auth`, `services/payment-api`
Accès privilégié	back-office : identifiant → visage (3 poses) → mot de passe + rôle, verrouillage 5 min (§7.1)	`services/{auth,face-id,backoffice}`, ADR 0007
Automate	voir §5 : proxy restreint, token, listes blanches, hors edge	`remediation/`, ADR 0002

Écarts assumés et documentés : pas de MFA pour les clients (seulement la confirmation renforcée des gros virements) ; secret JWT symétrique partagé entre `auth` et `payment-api` ; TLS auto-signé ; rate-limit en mémoire ; sauvegardes sur le même hôte.

7.1 Consigne Additionnelle 1 : accès privilégié au back-office

Reçue le 20/10, la consigne demande une authentification renforcée du back-office par reconnaissance faciale, avec un témoin LED piloté par un ESP8266. Nous l'avons traitée comme un **facteur supplémentaire sur l'accès privilégié**, jamais comme une dépendance de la plateforme : `verify_tx`, la détection, la remédiation et le Clip Chrono n'appellent ni la caméra, ni `face-id`, ni le broker (garde-fou D4, `docs/consigne-additionnelle-1.md`).

- **Parcours** : identifiant (réponse identique qu'il existe ou non) → **3 poses du visage dans un ordre tiré au hasard par le serveur** (face, gauche, droite), chaque image validée en pose **et** en identité par le modèle de l'équipe (`face_recognition`/dlib) → mot de passe et rôle. Deux échecs verrouillent 5 min le couple (identifiant, IP). Le visage n'est jamais un contournement du mot de passe.
- **Caméra de l'hôte** : le back-office et les routes `/auth/*`, `/admin/*` ne sont servis que par un listener du WAF lié à `127.0.0.1:9443` ; le serveur public répond 404 sur ces chemins.
- **Données biométriques** : 3 gabarits et 3 vignettes basse résolution par agent, **chiffrés AES-256-GCM** (clé SOPS, lié à l'identifiant), consultation journalisée, effacement total. C'est un amendement assumé de la règle « aucune image stockée », justifié par le réenrôlement (ADR 0007).
- **Habilitations** : rôles `customer/operator/admin`, enrôlement et révocation depuis le back-office, premier administrateur créé par un jeton de bootstrap, interdiction de retirer le dernier administrateur.
- **LED** : `auth` publie des états sémantiques sur MQTT (broker Mosquitto avec authentification et ACL par topic) ; le firmware ESP8266 rend les 5 motifs. Le client MQTT est *fail-open* : une panne du broker n'empêche pas de se connecter.

Le risque restant (3 poses rejouées dans l'ordre affiché, deepfake temps réel) est porté dans l'EBIOS (SO7–SO9). La procédure de secours si le facteur biométrique est perdu est décrite dans le PCA/PRA (§10).

8. Résultats des essais

Tests automatisés (hors stack, rejouables par `make test`) — un process pytest par composant puis Vitest : **1 667 tests verts au 22 octobre 2026** (7 e2e sautés hors stack).

Composant	Tests	Ce qu'ils prouvent
`services/auth` / `payment-api`	228 / 247	règles métier (virement atomique, solde, plafonds, double dépense concurrente), JWT, rate-limit, step-up, parcours à étapes et verrouillage du back-office, métriques et logs du contrat
`services/face-id` / `backoffice`	94 / 134	vérification par pose et identité (modèle simulé), chiffrement des gabarits, habilitations, capture guidée
`services/front` / `dashboard` (Vitest)	138 / 152	parcours bancaires D1–D16 ; enveloppe SSE réelle du moteur, lecture Prometheus, IP bloquées
`ml/*`	108	vecteur de features figé, entraînement reproductible, calibration, confirmation sur fenêtres, IP source, événement conforme

Composant	Tests	Ce qu'ils prouvent
`remediation`	176	invariants de la table, séquences par type, vérification d'abord, cooldown/idempotence/escalade, listes blanches, expiration des blocages, jeton opérateur, **exécution réelle du playbook WAF**, chemin SOC
`security/waf`, `attacks`, `infra`, `scripts`, libs	50 / 36 / 76 / 65 / 31	contrats de configuration (Dockerfile, nginx, compose, Prometheus, SOC, listener hôte), scénarios Red/Blue, seed et `verify_tx`, banc de mesure, packaging sans secret
`assistant`	132	indexation, réponse, résumé DSI, post-mortem, aucune capacité d'action, mode dégradé sans Ollama
`tests/e2e`	7 (sautés hors stack)	rejeu des 5 temps contre la stack réelle, dont `data_corruption` KO → OK

Essais sur la stack (21–22 octobre). Le banc `make bench` a d'abord établi la référence : 25 à 37 s (p95) entre l'attaque et l'`AnomalyEvent`, et un flood mal routé. Les rejeux ont surtout révélé qu'**aucun incident n'avait jamais pu être clos** : `verify_tx` visait le mauvais port, Ansible n'avait pas de dossier temporaire, le proxy Docker n'atteignait pas le socket, et l'IP de l'attaquant était masquée par Docker Desktop (le moteur a même bloqué sa propre IP). Après correction, la boucle a été rejouée de bout en bout : flood détecté en 2 s puis incident clos, `auth_abuse` depuis une IP du hotspot (mode « IP réelle ») détecté en 4 s, IP bloquée et incident clos en 4,2 s sans gêner les autres clients. La tolérance aux pannes a aussi été éprouvée : **188 requêtes sur 188 servies** en tuant une réplique de `payment-api`. Le Clip Chrono est outillé pour une prise unique sans coupe (`docs/dossier/clip/`).

Scénario	Mesuré (rejeu du 21/10)	Délai de détection p95	MTTR p95	Cible (RTO perçu)
service_flood	détecté en 2 s, clos en 1,0 s	[M]	[M]	≤ 60 s
auth_abuse	clos en 1,3 s (4,2 s en mode IP réelle)	[M]	[M]	≤ 60 s
web_attack	[M] (règle modifiée le 22/10)	[M]	[M]	blocage immédiat
data_corruption	[M]	[M]	[M]	≤ 2 min

Règle de rigueur : le MTTR du moteur part de la détection ; le RTO perçu ajoute le délai de détection (15 à 45 s attendus, une à trois collectes). Après ≥ 10 rejeux, `RTO retenu = ⌈1,5 × p95(D + MTTR)⌉` ; si la cible n'est pas tenue, c'est la cible que l'on révise, pas la mesure.

9. Analyse des risques (EBIOS Risk Manager)

Méthode appliquée en cinq ateliers, reliée à l'architecture réelle (`docs/ebios-rm.md`). **Valeurs métier** : le service de paiement (disponibilité), l'intégrité des transactions et des soldes, la capacité de reprise

autonome, la confiance dans les accès et les secrets. **Biens supports** : WAF, services, PostgreSQL et ses dumps, le moteur de remédiation (privilèges élevés par nature), la chaîne d'observabilité et de détection, les secrets SOPS, l'hôte. **Sources de risque retenues** : opportuniste (indisponibilité), fraudeur (comptes, failles web), attaquant ciblé (altération de données), initié négligent (secrets).

Coût de l'arrêt d'une fonction de paiement (hypothèses fictives) : environ 240 transactions/min à 0,25 € soit ~60 €/min de manque à gagner direct ; mais ce coût est faible devant les coûts indirects et réglementaires, qui croissent **par seuils de durée** (incident majeur DORA). Passer d'une reprise manuelle (minutes à heures) à une reprise automatique bornée (< 1 min) change la **classe** de l'incident.

Scénario opérationnel	Événement redouté	Risque initial	Risque résiduel	Mesure qui justifie la baisse
SO1 flood	indisponibilité	**12 Élevé**	6 Moyen	détection ML + blocage source + redémarrage, MTTR borné ; reste la rotation d'IP
SO2 abus d'authentification	prise de compte	**12 Élevé**	6 Moyen	rate-limit, blocage, durcissement ×0,5 ; reste l'absence de MFA
SO3 attaque web	altération / fuite	**12 Élevé**	8 Moyen	CRS, ORM paramétré, validation ; gravité inchangée (0-day)
SO4 corruption	altération	8 Moyen	6 Moyen	restauration vérifiée ; perte **bornée au RPO**
SO5 secret exposé	compromission de secret	**12 Élevé**	8 Moyen	SOPS, gitleaks, audit ; JWT symétrique reste critique
SO6 abus de l'automate	abus de privilèges	**12 Élevé**	**3 Faible**	proxy restreint, listes blanches, token ; un abus = redémarrages de 3 services
SO7 usurpation du visage	prise de compte privilégié	9 Élevé	6 Moyen	poses en ordre aléatoire, mot de passe obligatoire, listener réservé à l'hôte, verrouillage
SO8 fuite biométrique	fuite de données sensibles (RGPD)	**12 Élevé**	8 Moyen	chiffrement AES-256-GCM, clé hors base ; gravité inchangée (donnée non révocable)
SO9 facteur visage indisponible	back-office inaccessible	6 Moyen	4 Faible	paiement et reprise indépendants ; break-glass, deux

Scénario opérationnel	Événement redouté	Risque initial	Risque résiduel	Mesure qui justifie la baisse
				administrateurs enrôlés

Aucun risque élevé ou critique ne subsiste. Les risques moyens sont **proposés à l'acceptation de la direction** avec un indicateur de surveillance chacun (MTTR, échecs de login, blocages WAF, âge du dernier dump, résultat de `audit.sh`). Risques résiduels majeurs portés à la direction : détection calibrée sur quelques heures de trafic de démonstration ; corruption silencieuse non détectée par le signal retenu ; absence de retour arrière automatique des mesures (deny-list, rate-limit) ; le moteur est à la fois la meilleure défense et la cible de plus forte valeur.

10. Objectifs de continuité, PCA/PRA

Services critiques : paiement (vital), données (vital), WAF (vital, seule porte), authentification (haute), chaîne de reprise (haute pour l'autonomie, nulle pour le paiement : elle n'est **pas sur le chemin critique**), supervision et assistant (moyenne/faible).

Service / scénario	RTO cible (perçu)	RPO cible	Famille	Justification
Paiement — `service_flood`	**≤ 60 s** nominal, ≤ 5 min avant escalade	**0**	PCA	redémarrage sans restauration ; cohérent avec le Clip Chrono
Authentification — `auth_abuse`	≤ 60 s (login dégradé)	0	PCA	blocage + appel HTTP interne
Données — `data_corruption`	**≤ 2 min** sur le jeu de démo	**≤ 5 min** (dump périodique)	**PRA données**	la restauration est la seule réponse valable
WAF	≤ 30 s	0 (configuration versionnée)	PCA	reload / restart
Chaîne de reprise	≤ 5 min (manuel)	0 (modèle versionné)	—	hors chemin critique
Sinistre majeur (perte de l'hôte)	≤ 30–60 min (reconstruction)	≤ 1 h	PRA	**manuel, non éprouvé** : la reprise n'est plus zero-touch
Accès back-office — perte du facteur visage	minutes (redéploiement `auth` en break-glass)	dernier enrôlement	PRA accès	**manuel, non éprouvé** ; mot de passe et rôle restent exigés ; clé biométrique sauvegardée hors hôte (`docs/pca-pra.md` §5.7)

Procédures : automatiques pour les quatre scénarios (séquences §5) ; manuelle en mode dégradé (escalade, panne du moteur ou du proxy, type unknown) — l'opérateur applique la même séquence que l'automate puis `make verify`, et l'incident perd son caractère zero-touch. **Responsabilités** (matrice RACI dans `docs/pca-pra.md` §6) : l'humain **décide avant** (table de décision versionnée, RTO/RPO,

acceptation des risques), l'automate **exécute pendant**, l'humain **analyse après** (post-mortem, levée des mesures).

Choix de restauration et risques qui subsistent après la reprise : la restauration automatique de `latest` convient à la démo ; en production nous recommandons une validation humaine avant une restauration destructive. Risques résiduels : perte des transactions jusqu'au RPO, non rejouables (le journal ne porte pas le montant) ; dump déjà corrompu restauré par `latest` (parade : rétention ≥ 12 dumps, contrôle d'invariants avant rotation) ; corruption silencieuse ; rotation d'IP ; mesures sans retour arrière ; points uniques de défaillance (hôte, WAF, Postgres, moteur) ; sauvegardes sur le même hôte. Deux conditions manquent pour que les chiffres soient exacts : le **job de dump planifié** et la **mesure** des MTTR en recette.

11. NIS2 / DORA et choix de gouvernance

Cadrage : entité financière fictive de type établissement de paiement → **DORA** (règlement 2022/2554) est le texte sectoriel applicable et prime sur NIS2 pour le risque TIC et la notification d'incident ; NIS2 (art. 21) sert de grille de contrôle croisée. Une seule chaîne d'incident, un seul registre de risques. Ce dossier **ne revendique aucune conformité** : il dit ce que le prototype prouve, ce qu'il ne prouve pas, et ce que nous recommandons.

Exigence	Preuve dans le démonstrateur	Niveau
Détection d'activités anormales (DORA art. 10)	modèle ML versionné, seuil calibré, métrique et événement de détection	Démontré (trafic de démonstration)
Journalisation et traçabilité (art. 9–10)	logs JSON corrélés par `request_id`/`incident_id`, journal et timeline d'incident horodatés	Démontré
Réponse et reprise (art. 11)	remédiation autonome vérifiée par transaction, escalade, distinction disponibilité/intégrité	Démontré (recette [M])
Sauvegarde et restauration (art. 12)	restauration idempotente par playbook ; dump planifié et copie hors hôte manquants	Partiel
Tests de résilience (art. 24–25)	banc d'essai à 4 scénarios + chaos ; pas de programme annuel ni de TLPT	Partiel
Contrôle d'accès, moindre privilège, segmentation (art. 9)	WAF seul exposé, réseaux, proxy restreint, non-root ; accès privilégié à facteurs multiples (visage + mot de passe + rôle), confirmation des gros virements ; pas de MFA client	Partiel
Gouvernance par la direction (art. 5), reporting réglementaire (art. 19), tiers (art. 28–30), TLPT (art. 26)	non démontrables en 4 jours — dit explicitement ; le journal capture les champs d'un rapport d'incident	Non démontrable

Arbitrages recommandés à une direction (chacun fondé sur un élément vérifiable du dépôt) :

	Décision	Recommandation
D1	Automatiser la reprise ?	**Hybride** : automatique pour les mesures réversibles et bornées ;

	Décision	Recommandation
		validation humaine avant restauration destructive en production (la démo reste 100 % automatique)
D2	Quel RPO financer ?	démo : dump 5 min ; production : **WAL archiving + réplica** (RPO proche de 0 sur des données financières)
D3	ML ou règles ?	ML + garde-fou documenté ; ré-entraîner sur 2–4 semaines de trafic réel, processus de revue du seuil
D4	Droits de l'automate ?	proxy restreint, jamais le socket brut ; trace d'audit immuable, revue des listes blanches
D5	SIEM ou SOC intégrée ?	démo : SOC intégrée (16 Go) ; production : SIEM avec rétention et détection 24/7
D6	Délais de notification ?	écrire et répéter la procédure (initiale, intermédiaire, finale) alimentée par le journal
D7	Secrets ?	démo : SOPS ; production : Vault/HSM, rotation, **JWT asymétrique**
D8	Par où commencer la gouvernance ?	registre des tiers, approbation du cadre TIC, MFA, formation, audit indépendant

Feuille de route : **30 jours** (responsable DORA, procédure de notification, dump planifié + copie hors hôte + test de restauration, journaliser montant/comptes, retour arrière des mesures) ; **90 jours** (registre des tiers, MFA, SIEM, PITR, ré-entraînement sur trafic réel) ; **180 jours et plus** (programme annuel de tests, cadrage TLPT, formation des dirigeants, revue indépendante).

12. Limites, et ce que nous ferions différemment en contexte industriel

- **Un seul hôte, un seul WAF, un seul Postgres** : contrainte du sujet, assumée en risque résiduel ; en industriel, réplication et second site.
- **Détection** calibrée sur une heure de trafic de démonstration ; type d'incident par heuristique. En industriel : trafic réel, dérive surveillée, revue du seuil, et un modèle par service.
- **Restauration automatique destructive** : acceptable en démo, à encadrer par une validation humaine en production (D1).
- **Mesures sans expiration** (deny-list, rate-limit) : à corriger (expiration, dédoublonnage côté `auth`) ; la levée reste une décision humaine post-incident.
- **Pas d'intégration continue** pendant le sprint (décision d'équipe pour ne pas ralentir les livraisons) ; `make test` et `audit.sh` sont prêts à être branchés.
- **Recalibrage ML** : le modèle a été calibré avant les 3 répliques et la limitation de débit du WAF ; le seuil de démo (0,65) est un choix d'équipe plus sensible que la calibration (0,708). Recapture à refaire sur la stack définitive (ROADMAP T1).

- **Consigne Additionnelle 1 non recettée** : le build de dlib, la caméra réelle, le calibrage des marges de pose, l'ESP8266 et le broker n'ont été validés qu'en tests unitaires et de contrat ; les e2e du parcours facial (lot L10) restent à écrire. Aucun livrable noté n'en dépend.
- **Validation live** : plusieurs ajouts (carte IP bloquées, `make unblock`, mode « IP réelle » avec Caddy à installer sur le PC hôte) sont testés hors Docker ; la boucle de reprise elle-même a été rejouée sur la stack (§8).
- **Secret opérateur** : le jeton de déblocage n'est pas encore dans le fichier SOPS chiffré ; il est généré localement en attendant (ROADMAP T2).

13. Contribution de chacun

Le suivi individuel repose sur les commits (`type(scope): sujet`, `git shortlog -sn`), sur `ROADMAP.md`, qui donne pour chaque tâche sa preuve (commit ou test), et sur les déclarations de l'équipe pour le travail sans commit à son nom. État au 22 octobre 2026.

Membre	Domaine(s)	Contributions identifiables dans le dépôt
Yanis Capelle	Développement, Transformation, intégration	contrats d'interfaces, `payment-api` et refonte de l'app bancaire (front D1–D16), `auth`, moteur de remédiation, WAF, base de données, dashboard ; service `face-id` et back-office (première version) ; EBIOS RM, PCA/PRA, NIS2/DORA ; revues et intégration des branches, dossier et support
Valentin Furcy	IA / Data, Infrastructure	détection ML sur séries réelles (captures, calibration, détection en quelques secondes, IP source), modèle facial de l'équipe (notebook), chaîne `web_attack` bout en bout, banc de mesure, fiabilisation du lancement de la stack, 3 répliques de `payment-api`, mode « IP réelle » (Caddy), carte « IP bloquées »
Louis Nectoux	Cybersécurité	scénarios Red Team et console d'attaque, runbook PC Attaquant, durcissement et `audit.sh`, règles SOC, chaos engineering, tests e2e ; authentification faciale du back-office de bout en bout (parcours à étapes, habilitations, PR #53), secrets locaux du back-office
Enzo Palermo	Infrastructure, IoT, Cybersécurité	configuration SOPS + age, firmware ESP8266 de la LED (Wi-Fi, HTTP, MQTT), broker Mosquitto durci (profil `iot`),

Membre	Domaine(s)	Contributions identifiables dans le dépôt
		routage WAF de l'API d'authentification
Aurélien Serre	IA / Data	assistant RAG intégré : onglet du dashboard, export du journal d'incident, résumé DSI, post-mortem, proposition de commande opérateur avec confirmation, réseau `ai` isolé du moteur
Léo Solana	Développement, outillage	configuration de l'environnement (`.env`), `Makefile` (commandes de lancement, de test et de démo), revue des branches avant fusion

Une partie du code et de la documentation a été produite avec l'aide d'assistants d'IA générative (sujet §5) ; chaque contribution a été relue, testée et intégrée par l'équipe, qui en assume la maîtrise devant le jury.

Annexe A — Où vérifier (dépôt `Workshop2026-M2-G1-Code.zip`)

Affirmation	Commande / fichier
Déploiement reproductible	`make decrypt && make up` (profil démo) ; `make up-full` (observabilité complète) ; `infra/ansible/playbooks/deploy.yml`
Transaction de contrôle	`make seed && make verify` (`scripts/verify_tx`, code 0 = OK)
Démo en 5 temps	`make demo SC=service_flood` ; attaque depuis le PC Attaquant : `attacks/RUNBOOK-PC-ATTAQUANT.md`
Tous les tests	`make test` (procédure venv : README § Tests)
Durcissement, exposition, secrets	`security/hardening/audit.sh` ; `gitleaks detect --source .`
Contrats d'interfaces	`docs/contracts.md` ; invariants : `remediation/decision.py`, `infra/tests/`, `security/waf/tests/`
Modèle ML	`make capture-baseline`, `make capture-attack`, `make calibrate`, `make train` (`ml/training/README.md`)
Mesure de la détection	`make bench` (`scripts/bench_detection.py`) ; calibration : `ml/training/data/calibration.json`
Consigne Additionnelle 1	`docs/consigne-additionnelle-1.md`, `docs/RUNBOOK-BACKOFFICE-FACIAL.md`, ADR 0004–0007
Analyses de gouvernance (versions longues)	`docs/ebios-rm.md`, `docs/pca-pra.md`, `docs/nis2-dora.md`, `docs/adr/`
Avancement et contributions	`ROADMAP.md`

Annexe B — Glossaire

MTTD : délai entre la détection et la première action de remédiation. **MTTR** : délai entre la détection et la vérification réussie (`verify_tx` OK). **RTO perçu** : délai de détection + MTTR. **RPO** : perte de données maximale admise (âge du dernier dump). **AnomalyEvent** : objet JSON du contrat §3 émis par le détecteur ou l'adaptateur SOC. **Table de décision** : `remediation/decision_table.yml`, lien incident → actions. **Zero-Touch Recovery** : reprise sans intervention corrective humaine entre l'incident et la preuve de retour à la normale.